AI Wordle Solver

Academic Project Report

B.Tech Computer Engineering

Shubh Lineswala

2026

Institution name, roll number, faculty supervisor, and submission date are not recorded in the project repository and are left blank for the student to complete.

A standalone Python desktop solver using constraint satisfaction and Shannon entropy.
https://github.com/shubh-1225/AI-Wordle-Solver

Abstract

Wordle is a six-turn word-guessing game in which each guess receives a five-cell green, yellow, or grey pattern that constrains the hidden answer. This project implements a standalone Python desktop solver that treats that pattern as a constraint on the original Wordle answer list and recommends the next guess by Shannon entropy. A candidate remains feasible only if the project's two-pass feedback engine would have produced exactly the observed colouring, including correct handling of repeated letters. Allowed guesses, including words that are not possible answers, are ranked by how evenly they partition the remaining answers. The software comprises a CustomTkinter interface for assisted play, an autonomous solver mode, a batch simulator, and an optional precomputed uint8 feedback matrix. Evaluation used the original 2,315-answer / 12,972-guess dictionaries. Under a greedy maximum-entropy policy, the five highest-entropy opening words each solved every answer within six guesses, with mean game lengths between 3.437 and 3.472. Highest first-guess entropy (soare, 5.886 bits) was not the lowest full-game average (reast, 3.437). The policy is greedy one-ply ranking, not a proven minimax optimum, and the results apply only to these word lists.

1. Introduction and Problem Statement

Wordle asks a player to identify a hidden five-letter English word in at most six guesses. After each guess the game colours every letter green (correct position), yellow (present elsewhere), or grey (not usable under Wordle's duplicate-letter rules). The colouring is a partition of a large discrete search space: originally 2,315 possible answers and 12,972 allowed typed guesses.

Random or purely intuitive guessing wastes turns on words that distinguish few remaining answers. A useful next guess is one that, in expectation, splits the feasible set as evenly as possible so that later turns face a smaller candidate pool. The engineering problem is to implement official colouring, keep only consistent answers, score guesses by information, and measure full-game performance.

Problem statement. Design and implement a local desktop system that (i) colours guesses according to Wordle's two-pass rules, (ii) maintains the set of remaining answers as a constraint-satisfaction state, (iii) recommends high-information guesses from the allowed vocabulary, and (iv) reports reproducible solver statistics on the original answer list. The work is not an LLM client and does not claim optimality over all possible strategies or over the live New York Times vocabulary.

Scope. The delivered system includes an interactive board and keyboard, entropy and expected-remaining ranking, an optional NumPy feedback cache, autonomous simulation, and opener benchmarks. Multi-ply lookahead, reinforcement learning, hard mode, and a live vocabulary feed are not implemented.

2. Background and Methodology

Feedback. Each cell is encoded as 0 (grey), 1 (yellow), or 2 (green). Greens are assigned first; unused answer letters then supply yellows. Extra copies of a letter that cannot be matched become grey. This two-pass procedure is required for words such as sassy and alloy and is implemented in app/solver/feedback.py. Tests lock the colouring independently of the GUI.

Constraints and filtering. After observation (g, p), the feasible set is C = { a in Answers | feedback(g, a) = p for every prior observation }. There is no separate letter-count propagator: replaying the colouring function is the constraint store. The same guess with two different patterns raises ContradictoryFeedbackError. Filtering starts from all 2,315 answers and retains order.

Entropy ranking. A guess g maps each remaining answer to one of 243 = 3^5 patterns. If pattern i occurs for n_i of N candidates, p_i = n_i / N and

$$H(g) = -\Sigma_i \, p_i \, log_2(p_i)$$

where g is a candidate guess, i indexes nonempty feedback patterns, and p_i is the fraction of remaining answers that would produce pattern i. Ranking uses $H(g)$ under maximum entropy, or expected remaining size under candidate reduction.

H(g) is the expected information in bits. The solver also records the number of nonempty groups and the expected remaining size E[|C'|] = sum_i n_i^2 / N. Expected fractional reduction is 1 − E[|C'|]/N.

Two greedy objectives are implemented in app/solver/ranking.py and app/solver/simulator.py: maximum_entropy (sort by −H, then prefer a remaining answer, then alphabetical) and candidate_reduction (sort by E[|C'|], then −H, with the same tie-breaks). When one candidate remains, that word is guessed. Play is one-ply greedy; there is no lookahead tree.

Precomputation. Each pattern is stored as a base-3 integer in {0, …, 242}. scripts/precompute_patterns.py builds a uint8 matrix of shape 12,972 × 2,315 (about 29 MiB), fingerprinted with SHA-256 of both word lists. Ranking then uses numpy.bincount on the relevant submatrix. The cache is generated locally and is not required for correctness.

3. System Design and Implementation

The implementation is Python 3.11+ with CustomTkinter for the window, NumPy for the cache and histogram counts, and Pytest for unit and smoke tests. The GUI never computes entropy; GameSession calls WordleSolver. Ranking of the full guess pool runs on a worker thread so the window stays responsive.

Assisted play. The user types a five-letter guess (physical or on-screen QWERTY), presses Enter, cycles tile colours grey → yellow → green, and submits. The session applies the observation, filters candidates, and displays the best guess, entropy, group count, expected reduction, a short list of alternatives, a scrollable filtered candidate list, and compact game statistics (guess number, remaining size, cumulative information, status).

Solver mode plays a hidden answer (typed or randomly chosen from the answer list) with a selected strategy and optional opener, animating turns on the board. Simulation mode runs a background batch and writes aggregate statistics into the same panel; those session figures are not a substitute for the published 2,315-answer opener JSON.

Word lists are loaded from data/answers.txt and data/allowed_guesses.txt with validation (lowercase, length five, ASCII alphabetic, unique, answers ⊆ guesses). Invalid guesses and contradictory feedback are rejected with status messages rather than silent state corruption.

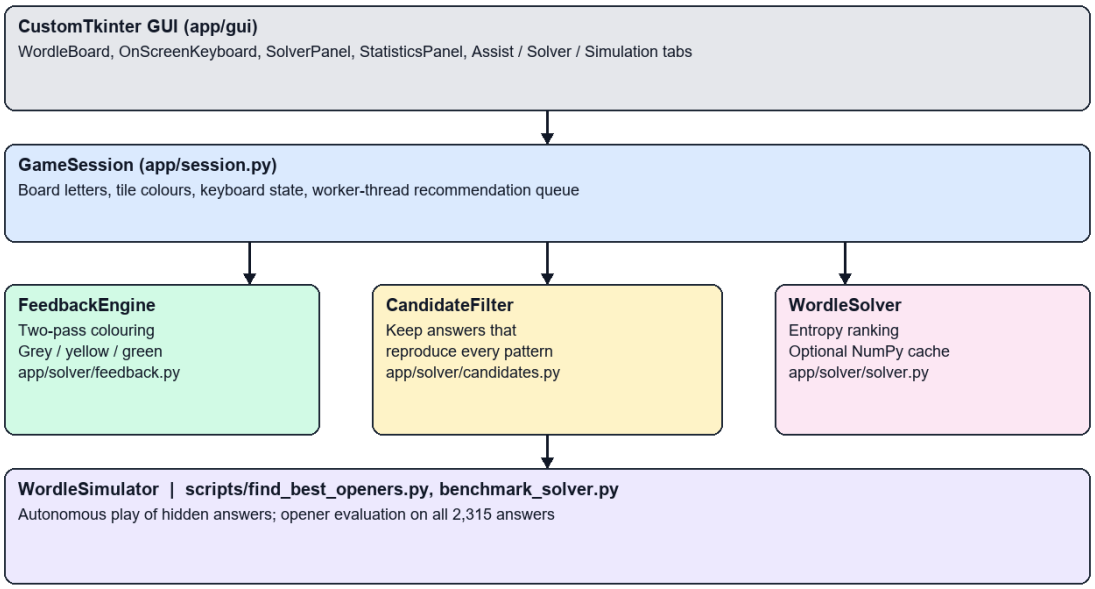


Figure 1. Layered architecture. The GUI calls GameSession only; colouring, filtering, and ranking reside in app/solver/.

4. Experimental Evaluation and Results

Unless noted, numbers below are taken from benchmarks/results/opener_results.json, generated 2026-10-04T21:23:39Z. Procedure: python scripts/find_best_openers.py --top-entropy 5 --strategy maximum_entropy after building the feedback cache. Each selected opener is played as guess 1 against every one of the 2,315 answers; later guesses use maximum entropy. Failures would count as 7 guesses; none occurred. The run is deterministic for these lists, this strategy, and these openers. No random seed is required.

The five openers are the allowed words with highest first-guess Shannon entropy: soare, roate, raise, raile, and reast. Wall-clock time was about 93–95 seconds per opener on the machine that wrote the JSON. An independent simulate() sweep of all answers with opener soare reproduced the histogram 44 / 1,216 / 991 / 63 / 1, confirming that grouped opener evaluation matches per-answer simulation.

Rank	Opener	Answer?	H■	Mean	Med.	Worst	Win	Time (s)
1	reast	no	5.865	3.437	3	5	100%	92.9
2	soare	no	5.886	3.465	3	6	100%	93.7
3	roate	no	5.883	3.467	3	5	100%	94.5
4	raile	no	5.866	3.469	3	5	100%	94.6
5	raise	yes	5.878	3.472	3	6	100%	93.9

Table 1. Maximum-entropy full games on 2,315 answers. Source: benchmarks/results/opener_results.json (2026-10-04).

Guess-count histogram for soare: 0 / 44 / 1,216 / 991 / 63 / 1 games in 1–6 guesses; zero failures. raise is the only opener in this set that is itself an answer (one 1-guess win).

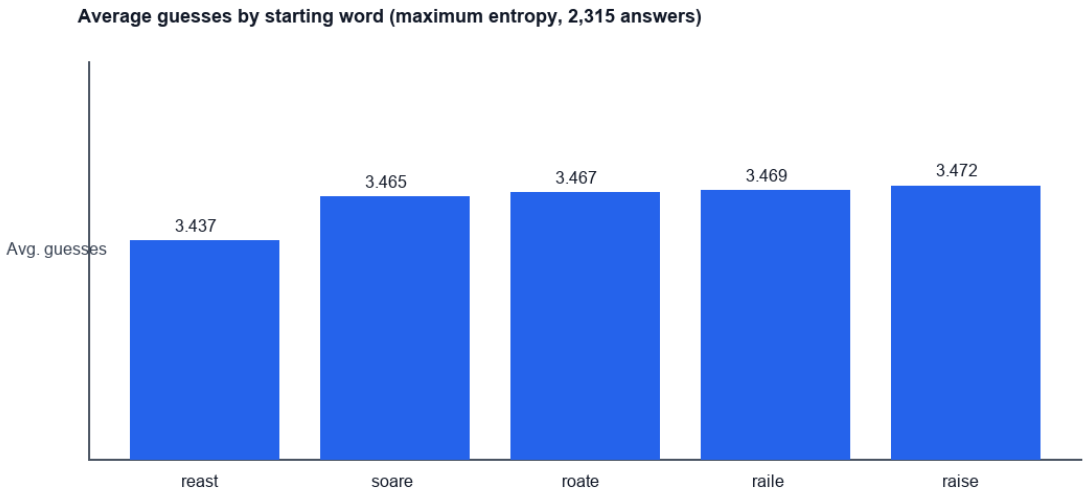


Figure 2. Mean guesses from Table 1. Highest first-guess entropy (soare) is not the lowest mean length (reast).

The candidate_reduction strategy is implemented and selectable in the GUI and scripts, but this report does not include a full 2,315-answer table for it; that comparison remains future work. Opening metrics for soare on the full answer list (live solver, cache enabled) are H = 5.886 bits, 127 nonempty groups, and expected remaining size 62.3 of 2,315 (about 97.3% expected reduction). After feedback of soare versus hidden answer crane (grey, grey, green, yellow, green), 19 candidates remain and the next recommendation is diact (2.971 bits).

Reproduction: python scripts/precompute_patterns.py ; python scripts/find_best_openers.py --top-entropy 5 --strategy maximum_entropy ; python scripts/export_benchmark_artifacts.py. These figures are not a claim of a universal Wordle optimum.

5. Conclusion and Future Work

The project delivers a correct two-pass Wordle engine, a candidate filter that is identical to that engine, greedy entropy and expected-remaining ranking over the allowed vocabulary, a desktop interface, and a reproducible full-dictionary evaluation. On the original 2,315 answers, maximum entropy with any of the five high-entropy openers won every game within six guesses, with mean length near 3.45. Non-answer probes occupied four of the five opener slots, and the best opening entropy was not the best mean game length.

Limitations. Results apply only to the original Wordle lists, not the live New York Times list. The policy is greedy one-ply ranking and is not a proven optimum. Hard mode is absent. The published strategy comparison covers maximum entropy only. The optional cache is large and machine-local.

Future work. Multi-ply expected remaining-guesses search; a published candidate_reduction sweep on the same 2,315 answers; optional learned policies on top of the same feedback function; and a maintained vocabulary adapter. GPU acceleration is not motivated until a deeper search is in place: one-ply ranking is already practical with the uint8 matrix.

References

[1] C. E. Shannon, “A Mathematical Theory of Communication,” Bell System Technical Journal, vol. 27, no. 3–4, pp. 379–423 and 623–656, 1948.

[2] J. Wardle, Wordle (original web game), 2021; subsequently published by The New York Times, https://www.nytimes.com/games/wordle (accessed Oct. 2026).

[3] cfreshman, original Wordle answer list (alphabetical). GitHub Gist, https://gist.github.com/cfreshman/a03ef2cba789d8cf00c08f767e0fad7b (retrieved 5 Oct. 2026).

[4] cfreshman, original Wordle extra allowed guesses. GitHub Gist, https://gist.github.com/cfreshman/cdcdf777450c5b5301e439061d29694c (retrieved 5 Oct. 2026).

[5] C. R. Harris et al., “Array programming with NumPy,” Nature, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[6] T. Schimansky, CustomTkinter documentation, https://customtkinter.tomschimansky.com/ (accessed Oct. 2026).

[7] pytest documentation, https://docs.pytest.org/ (accessed Oct. 2026).

[8] S. Lineswala, AI Wordle Solver, GitHub repository, https://github.com/shubh-1225/AI-Wordle-Solver (version 1.0.0).